AWS DEVOPS PRACTICAL ASSESSMENT

Secure Employee Management API

Implementation report, step-by-step build log, troubleshooting guide and interview preparation

Internet → CloudFront → AppSync → Application Load Balancer → ECS Fargate → RDS PostgreSQL

Candidate	**Akash Ajay**
Repository	github.com/AkashAjayInApp/employee-management-api
AWS region	us-east-1 (N. Virginia)
Application	Node.js 22 · Express 5 · PostgreSQL (pg)
Infrastructure method	AWS Console + AWS CLI (manual provisioning was permitted by the assessment brief)
CI/CD	GitHub Actions with GitHub OIDC → AWS IAM role (no stored keys)
Document date	14 October 2026
Note	This document contains AWS resource identifiers but no secrets. The AppSync API key and database password are intentionally omitted.

Contents

1 Executive summary
2 Architecture
3 Application (REST API)
4 Docker
5 Amazon ECR
6 Networking (VPC)
7 Security groups
8 Database & Secrets Manager
9 ECS Fargate
10 Application Load Balancer
11 AWS AppSync (GraphQL)
12 CloudFront
13 Monitoring & alerting
14 CI/CD pipeline
15 Security approach
16 Step-by-step build log
17 Problems encountered & how they were fixed
18 Troubleshooting guide
19 Operations: demo, pause/resume, cost
20 Requirements traceability
21 Known limitations & next improvements
22 Interview questions & answers

1 Executive summary

I designed, built and deployed a **Secure Employee Management API** on AWS that follows the required request flow **CloudFront → AppSync → ALB → ECS Fargate → Database**. The REST API (CRUD for employees plus a `/health` endpoint) runs as two containers on ECS Fargate in private subnets, stores data in Amazon RDS for PostgreSQL, and is exposed publicly only through CloudFront and an AppSync GraphQL API protected by an API key.

Area	What was delivered
Application	Node.js/Express REST API: `GET/POST /employees`, `GET/PUT/DELETE /employees/{id}`, `GET /health`. Input validation, graceful shutdown, pluggable storage (PostgreSQL in AWS, in-memory locally), 9 automated unit tests.
Docker	Hardened `node:22-alpine` image: non-root user, health check, OS patching, npm removed from runtime, RDS CA bundle for verified TLS. `.dockerignore` provided.
ECR	Repository `employee-api`; semantic tags for manual releases, git commit SHA tags from CI; scan-on-push enabled.
Networking	Dedicated VPC `10.0.0.0/16`, 2 public + 2 private subnets across 2 AZs, Internet Gateway, NAT Gateway, route tables.
Security groups	Chained least privilege: Internet → `emp-alb-sg` (80) → `emp-ecs-sg` (8080) → `emp-rds-sg` (5432).
Compute	ECS cluster `employee-cluster`, service `employee-api-service`, 2 Fargate tasks, rolling deployments, circuit breaker with automatic rollback.
Load balancing	Internet-facing ALB `emp-alb`, target group `emp-tg` (IP targets, `/health` checks).
GraphQL	AppSync API `employee-api-graphql` (public, API-key auth) with HTTP data source to the ALB and JavaScript resolvers for `employees` and `employee(id)`.
Edge	CloudFront distribution in front of AppSync: HTTPS redirect, caching disabled for GraphQL POST, correct header forwarding.
Database & secrets	RDS PostgreSQL `employee-db` (private, encrypted); master password managed by AWS Secrets Manager and injected into ECS at task start.
Monitoring	CloudWatch Logs, dashboard (ECS CPU/memory, ALB requests/5xx/latency percentiles/target health, logs), 3 alarms → SNS email.
CI/CD	GitHub Actions: Build → Unit test → Docker build → Trivy scan → Push to ECR → Deploy to ECS → Verify. AWS access via OIDC; no long-lived credentials.

Verified end to end

A GraphQL query sent to `https://d1a7kv1va3r66w.cloudfront.net/graphql` travels CloudFront → AppSync → ALB → ECS → RDS and returns employee data. Requests without an API key are rejected with HTTP 401. The latest version was deployed by the pipeline (task definition revision 5, registered by the GitHub Actions role, image tagged with commit `e207ac6`).

Key engineering decisions

- **Data in RDS, not in memory.** With two tasks, in-memory storage gives each task its own copy of the data, so a record created on one task is missing on the other. A shared database is required for correctness, not only for

persistence.

- **Liveness-only health check.** `/health` deliberately does not query the database, so a short database outage does not cause ECS to kill every task at once.
- **Secrets never in code.** ECS fetches the database credentials from Secrets Manager using the task execution role; the task definition only stores an ARN reference.
- **Security as a pipeline gate.** Trivy blocks images with fixable HIGH/CRITICAL vulnerabilities. This gate found and drove the fix for a real zlib CVE and 10 CVEs in npm's bundled packages.
- **Failure testing before release.** Deliberately stopping the database during local testing exposed a crash bug and a file-permission bug before they reached AWS.

2 Architecture

2.1 Request flow

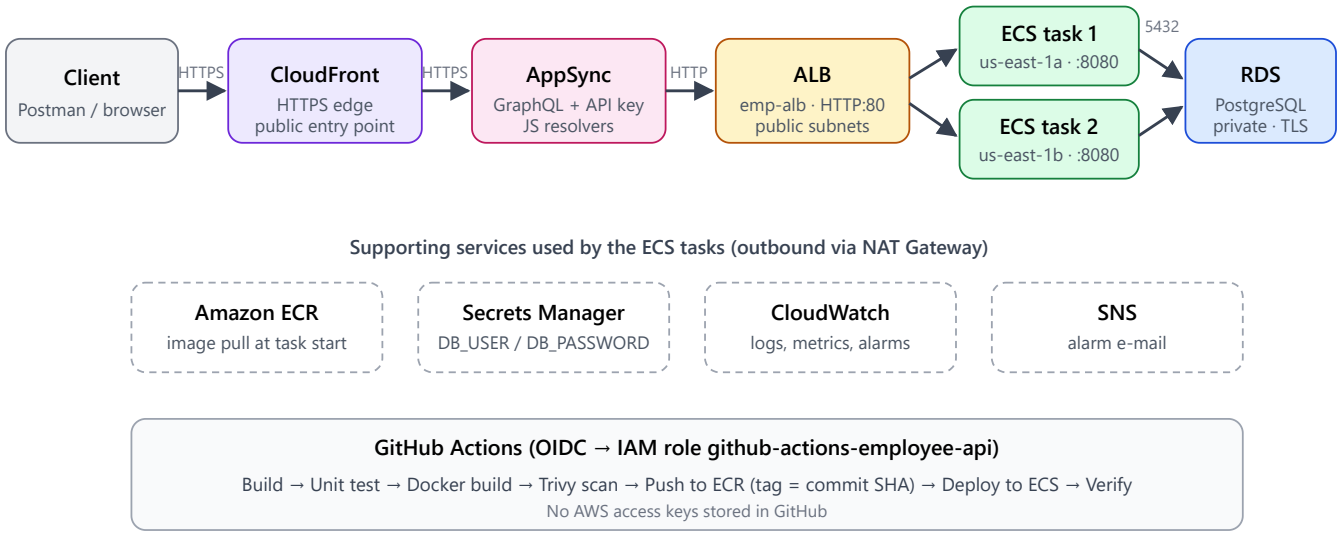


Figure 1 – Request flow and supporting services

2.2 Network layout

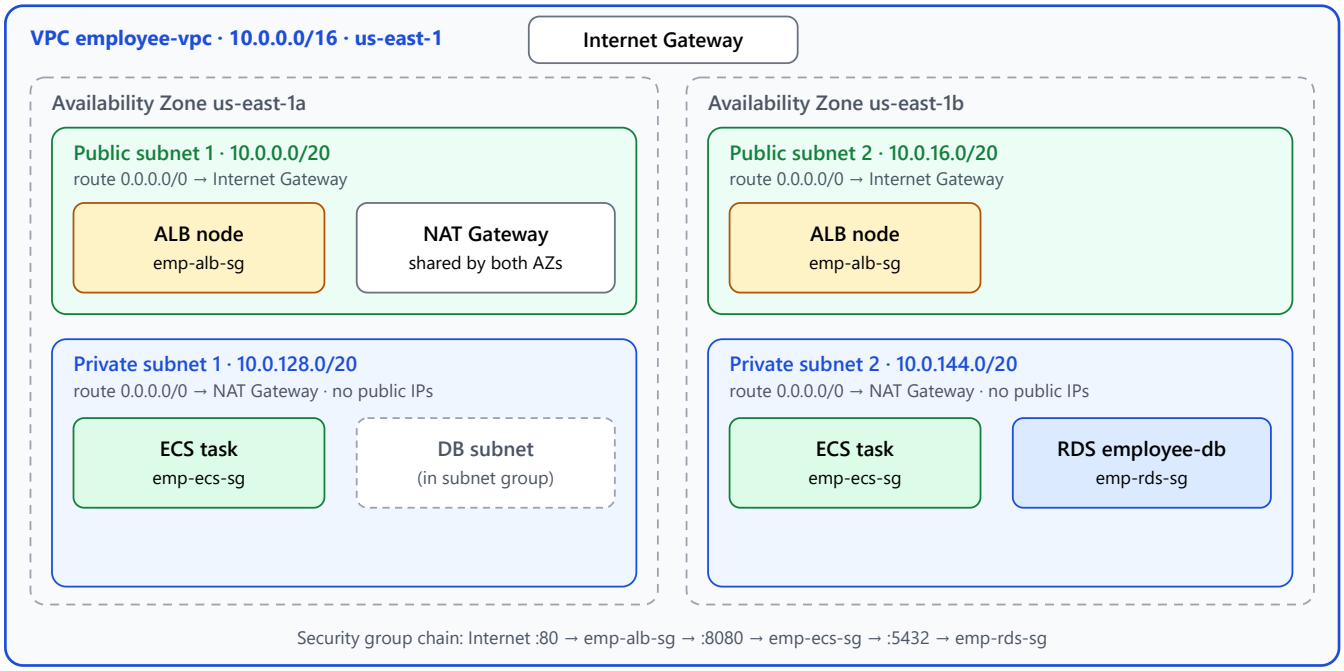


Figure 2 – VPC, subnets, routing and placement of each component

2.3 Resource inventory

Component	Name / ID	Key settings
CloudFront	`ERTV7AGSBNVMB` `d1a7kv1va3r66w.cloudfront.net`	Origin = AppSync host (HTTPS only, TLS 1.2), CachingDisabled, AllViewerExceptHostHeader, redirect HTTP→HTTPS

AppSync	`employee-api-graphql` `nxxt7677kzgodiyoift44v5vsa`	Public (GLOBAL) visibility, API_KEY auth, HTTP data source `employee_alb_source`
Application Load Balancer	`emp-alb` `emp-alb-820519526.us-east-1.elb.amazonaws.com`	Internet-facing, public subnets in 2 AZs, listener HTTP:80 → `emp-tg`
Target group	`emp-tg`	Target type IP, HTTP:8080, health `/health`, 2/2 thresholds, 5 s timeout, 15 s interval, deregistration delay 30 s
ECS	Cluster `employee-cluster` Service `employee-api-service`	Fargate, 2 tasks, rolling 100%/200%, circuit breaker + rollback, 60 s grace period, private subnets, no public IP
Task definition	`employee-task` (rev 5, deployed by CI)	0.25 vCPU, 512 MB, Linux/X86_64, awsvpc, container `employee-api`:8080, health check, awslogs
ECR	`employee-api`	Tags: `1.0`, `1.1.0`, `1.2.0`, commit SHAs; scan on push
RDS	`employee-db`	PostgreSQL 18.3, db.t4g.micro, 20 GB gp3, encrypted, single-AZ, not public, database `employees`
Secrets Manager	`rds!db-25993e51-…`	RDS-managed master credentials (keys `username`, `password`)
VPC	`employee-vpc` (`vpc-090621e891bfdcbb7`)	10.0.0.0/16, IGW `employee-igw`, NAT `nat-0f8e32a8b59057577`
Security groups	`emp-alb-sg`, `emp-ecs-sg`, `emp-rds-sg`	See section 7
IAM	`ecsTaskExecutionRole` `github-actions-employee-api`	Execution role (ECR pull, logs, one secret); CI role (ECR push, ECS deploy, scoped PassRole)
Monitoring	Dashboard `employee-api`; SNS `employee-alerts`	Alarms: `employee-ecs-cpu-high`, `employee-unhealthy-targets`, `employee-5xx-errors`; log group `/ecs/employee-task` (7-day retention)

3 Application (REST API)

The API is written in **Node.js 22 with Express 5**. It listens on port **8080**.

3.1 Endpoints

Method & path	Purpose	Responses
`GET /health`	Liveness check used by Docker, ECS and the ALB	200 `{"status":"UP"}`
`GET /employees`	List all employees (ordered by id)	200 array
`GET /employees/{id}`	Get one employee	200 object · 404 not found / invalid id
`POST /employees`	Create an employee	201 created · 400 missing/empty fields
`PUT /employees/{id}`	Replace an employee's name, department and email	200 updated · 400 invalid body · 404
`DELETE /employees/{id}`	Delete an employee	204 · 404
`GET /`	Welcome / status message	200

Employee object

```
{ "id": 1, "name": "Akash", "department": "QA", "email": "akash@example.com" }
```

All three fields (`name` , `department` , `email`) are required non-empty strings on POST and PUT. Any server-side failure (for example the database being unreachable) returns `500 {"message":"Internal server error"}` instead of hanging.

Example calls (via the ALB)

```
curl http://emp-alb-820519526.us-east-1.elb.amazonaws.com/health
curl http://emp-alb-820519526.us-east-1.elb.amazonaws.com/employees
curl -X POST http://emp-alb-820519526.us-east-1.elb.amazonaws.com/employees \
     -H "Content-Type: application/json" \
     -d '{"name":"Priya","department":"DevOps","email":"priya@example.com"}'
```

3.2 Code structure

```
src/
├── app.js            # routes, validation, error handler (createApp(store))
├── server.js         # chooses store, starts server, graceful SIGTERM shutdown
└── store/
    ├── memory.js     # in-memory store (local runs, unit tests)
    └── postgres.js   # PostgreSQL store (AWS), TLS with RDS CA bundle
test/employees.test.js  # 9 tests using Node's built-in test runner
Dockerfile · .dockerignore · .gitignore · .github/workflows/deploy.yml
```

3.3 Design decisions

Decision	Reason

Store abstraction (memory vs PostgreSQL)	Both stores expose the same functions (`list/get/create/update/remove`). If `DB_HOST` is set the app uses PostgreSQL; otherwise it uses memory, so `docker run -p 8080:8080 employee-api` works locally without a database and unit tests need no infrastructure.
Table created at startup	`CREATE TABLE IF NOT EXISTS employees` plus two seed rows if empty. The private database never has to be accessed manually.
Verified TLS to RDS	The AWS RDS CA bundle is baked into the image and `rejectUnauthorized: true` is used, so the app verifies it is talking to the real database.
`/health` does not touch the DB	It is a **liveness** check. If it checked the database, a brief DB outage would fail every task's health check and ECS would replace all tasks simultaneously, turning a small incident into a full outage.
Graceful shutdown on SIGTERM	ECS sends SIGTERM before stopping a task (deploys, scale-in). The server stops accepting new connections, finishes in-flight requests, closes the DB pool and exits 0, so rolling deployments cause no 5xx errors.
Pool `error` handler	Without it, the database dropping idle connections raised an unhandled error that crashed the process (found during failure testing).
IDs never reused	The original `length + 1` id logic produced duplicate ids after a delete; PostgreSQL `SERIAL` now guarantees uniqueness.

3.4 Unit tests

Run with `npm test` (Node's built-in `node --test`, no extra dependencies). The tests start the app on a random port with the in-memory store and cover: health, list, 404 on unknown id, 404 on non-numeric id, create, 400 on missing fields, ids not reused after delete, update, and delete-404. **Result: 9 passed, 0 failed.**

4 Docker

4.1 Dockerfile

```
FROM node:22-alpine
ENV NODE_ENV=production
RUN apk upgrade --no-cache                    # pull OS security fixes (e.g. zlib CVE)
WORKDIR /app
COPY package*.json ./                         # dependency layer cached separately
RUN npm ci --omit=dev && npm cache clean --force \
    && rm -rf /usr/local/lib/node_modules/npm /usr/local/lib/node_modules/corepack \
              /usr/local/bin/npm /usr/local/bin/npx /usr/local/bin/corepack
COPY src ./src
ADD --chmod=644 https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem /app/rds-global-
bundle.pem
USER node                                     # non-root
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD wget -qO- http://127.0.0.1:8080/health || exit 1
CMD ["node", "src/server.js"]                 # exec form: node is PID 1, receives SIGTERM
```

4.2 Why each line matters

Choice	Reason
`node:22-alpine`	Small, maintained official image; major version pinned for reproducible builds.
`apk upgrade`	Applies OS patches published after the base image was built. This fixed HIGH CVE in `zlib 1.3.2-r0 → r1`.
Copy `package*.json` before source	Docker layer caching: dependencies are reinstalled only when the manifests change.
`npm ci --omit=dev`	Exact versions from the lock file, production dependencies only.
Remove npm/corepack	The app runs with plain `node`. npm's own bundled packages carried 10 HIGH CVEs (pacote, sigstore, brace-expansion, ip-address, picomatch) that Trivy flagged. Removing them shrinks the attack surface.
RDS CA bundle with `--chmod=644`	Lets the non-root user read the certificate bundle used to verify the database's TLS certificate.
`USER node`	Requirement: avoid running as root. A compromise of the app does not give root inside the container.
`HEALTHCHECK`	Container-level liveness check (mirrored in the ECS task definition).
Exec-form `CMD`	Node runs as PID 1 and receives SIGTERM directly, enabling graceful shutdown.

4.3 .dockerignore

```
node_modules · npm-debug.log · test · .git · .github · .gitignore · terraform · *.md
Dockerfile* · .dockerignore · .env · .env.* · .vscode
```

Keeps the build context small and ensures local `node_modules`, tests, git history and any local secrets never enter the image.

4.4 Build and run locally

```
# In-memory mode (exactly as the assessment brief specifies)
docker build -t employee-api .
docker run -p 8080:8080 employee-api
curl http://localhost:8080/health          # {"status":"UP"}

# With a real PostgreSQL (two app containers sharing one DB, like 2 ECS tasks)
docker network create empnet
docker run -d --name emppg --network empnet -e POSTGRES_PASSWORD=localtest postgres:17-alpine
docker run -d --name empa --network empnet -p 8091:8080 -e DB_HOST=emppg -e DB_USER=postgres \
           -e DB_PASSWORD=localtest -e DB_SSL=false employee-api
docker run -d --name empb --network empnet -p 8092:8080 -e DB_HOST=emppg -e DB_USER=postgres \
           -e DB_PASSWORD=localtest -e DB_SSL=false employee-api
```

4.5 Local verification results

Test	Result
Container user	✓ `node` (not root)
Graceful stop (`docker stop`)	✓ "SIGTERM received, closing server", exit code 0
Write via container A, read via container B	✓ same data (shared database)
Database stopped	✓ `/employees` → 500, `/health` stays 200, process keeps running
Database restarted	✓ app recovers automatically without a restart
Trivy (HIGH/CRITICAL, fixable)	✓ 0 findings

5 Amazon ECR

Repository: `324440884610.dkr.ecr.us-east-1.amazonaws.com/employee-api`

5.1 Tagging strategy

Tag type	Example	Used for
Semantic version	`1.0`, `1.1.0`, `1.2.0`	Manual releases during the build-out. A new number for every change, never reused.
Git commit SHA	`e207ac649744f54e5af81e55a56a2112dcfe6012`	Every CI build. Each running container can be traced back to the exact commit that produced it.
`latest`	(legacy, not used for deployment)	Avoided: a moving tag means a task restarted tomorrow could pull different code than the one running today.

Rollback = point the ECS service at the previous task definition revision (which references the previous immutable tag).

5.2 Manual push (used for versions 1.1.0 and 1.2.0)

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS \
    --password-stdin 324440884610.dkr.ecr.us-east-1.amazonaws.com
docker tag employee-api:1.2.0 324440884610.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.2.0
docker push 324440884610.dkr.ecr.us-east-1.amazonaws.com/employee-api:1.2.0
```

> **Windows note**
> In Windows PowerShell 5.1 the piped `docker login` returned HTTP 400 because the password is re-encoded in the pipe. Running the same command in Git Bash works.

5.3 Vulnerability scanning

- **Scan on push** (registry scanning rule, BASIC, filter `employee-api`).
- Scan of the original `1.2.0` image: **1 HIGH** (zlib). Scan of the CI-built image: **0 findings**.
- Images built by Docker Desktop 29 were pushed as an OCI *image index*, which ECR basic scanning cannot scan directly. The pipeline builds with `--provenance=false --sbom=false` so a plain, scannable image manifest is pushed.

6 Networking (VPC)

Created with the VPC console's **"VPC and more"** option (2 AZs, 2 public + 2 private subnets, 1 zonal NAT Gateway, no VPC endpoints, DNS hostnames and resolution enabled).

Subnet	CIDR	AZ	Route table	Hosts
employee-subnet-public1-us-east-1a	10.0.0.0/20	us-east-1a	employee-rtb-public (0.0.0.0/0 → IGW)	ALB node, NAT Gateway
employee-subnet-public2-us-east-1b	10.0.16.0/20	us-east-1b	employee-rtb-public (0.0.0.0/0 → IGW)	ALB node
employee-subnet-private1-us-east-1a	10.0.128.0/20	us-east-1a	employee-rtb-private1 (0.0.0.0/0 → NAT)	ECS task, DB subnet group
employee-subnet-private2-us-east-1b	10.0.144.0/20	us-east-1b	employee-rtb-private2 (0.0.0.0/0 → NAT)	ECS task, RDS instance

6.1 Why this layout

- **Public subnets** have a route to the Internet Gateway, so the internet-facing ALB can receive traffic.
- **Private subnets** have no inbound route from the internet. ECS tasks have **no public IP** and cannot be reached directly.
- The **NAT Gateway** gives private tasks outbound-only internet access to pull images from ECR, read Secrets Manager and send logs to CloudWatch.
- **Two Availability Zones**: the ALB, the ECS tasks and the DB subnet group span both, so one AZ failure does not take the API down (subject to the single-NAT trade-off below).

6.2 NAT Gateway choice: Zonal, 1 AZ

Option	Assessment
None	Rejected – private tasks could not pull images or fetch secrets.
Regional	Works, but is billed per AZ in use (≈2× cost here).
Zonal, 1 AZ (chosen)	Cheapest (~$1/day). Trade-off: if us-east-1a fails, tasks in us-east-1b lose *outbound* internet access (inbound traffic via the ALB still works). Production would use one NAT per AZ or VPC endpoints.

7 Security groups



Each source is a security-group reference, not an IP range (except the public ALB rule).

Figure 3 – Least-privilege security group chain

Security group	ID	Inbound rule	Attached to
`emp-alb-sg`	sg-0ebb99fb27a3c57cb	HTTP 80 from 0.0.0.0/0	ALB
`emp-ecs-sg`	sg-0d51d0e2e48b28aad	TCP 8080 from **emp-alb-sg** only	ECS tasks
`emp-rds-sg`	sg-0ebd19f435650bf2f	TCP 5432 from **emp-ecs-sg** only	RDS

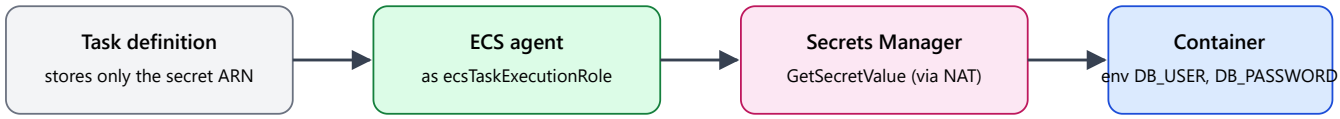
- **No 0.0.0.0/0 to the application port** (8080) – only the ALB's security group may reach it.
- Referencing security groups (instead of IP ranges) stays correct as ALB node IPs and task IPs change.
- Verified: a direct request to an ECS task's IP on port 8080 times out; the same request through the ALB succeeds.
- The earlier console-built setup shared the VPC's `default` security group between the ALB and ECS; this was replaced (see section 17).

8 Database & Secrets Manager

8.1 RDS PostgreSQL

Setting	Value
Identifier / engine	`employee-db` · PostgreSQL 18.3
Instance / storage	db.t4g.micro · 20 GB gp3 · storage autoscaling off · encrypted at rest
Initial database / master user	`employees` / `postgres`
Credentials	**Managed in AWS Secrets Manager** (secret `rds!db-25993e51-ff6d-42c1-8964-613c815d0f73`)
Network	VPC `employee-vpc`, subnet group `employee-db-subnet-group` (private subnets only), **public access: No**, SG `emp-rds-sg`
Availability / backups	Single-AZ (cost), automated backups 1 day
Endpoint	`employee-db.cutg6o4g6x3p.us-east-1.rds.amazonaws.com:5432`

8.2 How ECS retrieves the secret securely



The app then connects to RDS over TLS, verifying the server certificate with the RDS CA bundle.
The password never appears in GitHub, the Dockerfile, the image, the task definition or the console.

Figure 4 – Secret injection at task start

Task definition environment

Name	Type	Value
`DB_HOST`	Value	`employee-db.cutg6o4g6x3p.us-east-1.rds.amazonaws.com`
`DB_PORT`	Value	`5432`
`DB_NAME`	Value	`employees`
`DB_USER`	ValueFrom (secret)	`arn:aws:secretsmanager:…:secret:rds!db-25993e51-…-7A8x4Q:username::`
`DB_PASSWORD`	ValueFrom (secret)	`arn:aws:secretsmanager:…:secret:rds!db-25993e51-…-7A8x4Q:password::`

The `:username::` suffix format is `secret-ARN:json-key:version-stage:version-id`; empty stage and id mean "current version".

Execution role permission (inline policy `read-employee-db-secret`)

```
{ "Effect": "Allow",
  "Action": "secretsmanager:GetSecretValue",
  "Resource": "arn:aws:secretsmanager:us-east-1:324440884610:secret:rds!db-25993e51-ff6d-42c1-8964-
613c815d0f73-7A8x4Q" }
```

Verified with the IAM policy simulator: **allowed** for this secret, **implicitDeny** for any other secret. The secret uses the AWS-managed KMS key, so no extra `kms:Decrypt` permission is needed.

9 ECS Fargate

9.1 Task definition `employee-task`

Setting	Value
Launch type / network mode	FARGATE / awsvpc (each task gets its own ENI and private IP)
CPU / memory	0.25 vCPU / 512 MB · Linux / X86_64
Container	`employee-api`, essential, port 8080/TCP (HTTP)
Image	`employee-api:<commit-sha>` (set by the pipeline)
Container health check	`CMD-SHELL wget -qO- http://127.0.0.1:8080/health || exit 1` · interval 30 s · timeout 5 s · retries 3 · start period 10 s
Stop timeout	30 s (time between SIGTERM and SIGKILL)
Logging	awslogs → `/ecs/employee-task`, stream prefix `ecs`
Task execution role	`ecsTaskExecutionRole` (ECR pull, CloudWatch Logs, one secret)
Task role	None – the application makes no AWS API calls (least privilege)

Revision history: 1 (original, in-memory, default VPC) → 2 (image 1.1.0 + health check) → 3 (DB settings, key typo `DB_PORT5432`, deregistered) → 4 (typo fixed, image 1.2.0) → **5 (created by CI, image = commit SHA)**.

9.2 Service `employee-api-service`

Setting	Value
Desired tasks	**2** (spread across us-east-1a and us-east-1b)
Deployment	Rolling update, minimum healthy 100%, maximum 200%
Failure detection	Deployment circuit breaker **with rollback**
Health check grace period	60 s
Networking	Private subnets 1 and 2, `emp-ecs-sg`, **public IP disabled**
Load balancing	`emp-alb` → listener 80 → `emp-tg`, container `employee-api:8080`

9.3 Rolling deployment and recovery

- **100% / 200%**: ECS starts the new tasks first, waits for them to pass ALB health checks, then drains and stops the old tasks. Capacity never drops below 2.
- Measured during a rolling deploy: **200 probe requests, 200 × HTTP 200, 0 errors**, rollout completed in ~3 minutes.
- **Service recovery**: when a task is stopped or fails its health checks, the service scheduler notices running < desired and launches a replacement; the ALB registers it once healthy.
- **Circuit breaker**: if new tasks repeatedly fail to become healthy, the deployment is marked failed and ECS rolls back to the last working revision automatically.

9.4 Fargate vs ECS (assessment question)

	ECS	Fargate
What it is	The **orchestrator**: clusters, task definitions, services, scheduling, deployments, LB integration.	A **serverless compute engine** (launch type / capacity provider) that ECS can run tasks on.
Servers	With the EC2 launch type you manage EC2 instances: AMIs, patching, scaling, bin-packing.	No instances to manage. AWS provisions isolated compute per task.
Billing	Pay for EC2 instances (whether busy or idle).	Pay per task for requested vCPU and memory per second.
Why used here	Small, stateless service: Fargate removes server operations, gives per-task isolation and its own ENI (awsvpc) so security groups apply per task, and scales simply by changing the desired count.	

10 Application Load Balancer

Setting	Value
Name / scheme	`emp-alb` / internet-facing (AppSync reaches it over the internet)
Subnets	Public subnet 1 (us-east-1a) and public subnet 2 (us-east-1b)
Security group	`emp-alb-sg` only
Listener	HTTP:80 → forward to `emp-tg`
Target group	`emp-tg` : target type **IP** (required for Fargate/awsvpc), HTTP:8080, round robin
Health check	`/health` , success code 200, healthy 2, unhealthy 2, timeout 5 s, interval 15 s
Deregistration delay	30 s (default 300 s)
Target registration	Automatic – the ECS service registers/deregisters task IPs

Tuning rationale

- With 2 × 15 s, a new task receives traffic after ~30 s and a broken one is removed after ~30 s (defaults would take ~2.5 minutes to admit a new task).
- The API responds in milliseconds, so 30 s of connection draining is ample and makes every deployment ~4.5 minutes faster than the 300 s default.
- Two separate health checks with different jobs: the **ALB check** decides whether a task receives traffic; the **container check** decides whether ECS replaces the task.

> **Reading ALB errors**
>
> **503** from the ALB almost always means "no healthy targets". **502** means a target returned an invalid response or closed the connection. **504** means the target did not respond in time.

11 AWS AppSync (GraphQL)

Setting	Value
API	`employee-api-graphql` (`nxxt7677kzgodiyoift44v5vsa`)
Endpoint	`https://4mhjvucnqjhlja7ekiqknqa7ey.appsync-api.us-east-1.amazonaws.com/graphql`
Visibility	**Public (GLOBAL)** – required so CloudFront can use it as an origin
Authorization	API key (header `x-api-key`), valid 364 days
Data source	`employee_alb_source` , type HTTP, endpoint `http://emp-alb-820519526.us-east-1.elb.amazonaws.com`
Resolvers	Unit resolvers, runtime APPSYNC_JS 1.0.0, for `Query.employees` and `Query.employee`

11.1 Schema

```
type Employee { id: ID!  name: String!  department: String!  email: String! }

type Query {
  employees: [Employee!]!
  employee(id: ID!): Employee
}

schema { query: Query }
```

11.2 Resolver (employee by id)

```
import { util } from '@aws-appsync/utils';

export function request(ctx) {
  return {
    method: 'GET',
    resourcePath: `/employees/${util.urlEncode(ctx.args.id)}`,
    params: { headers: { 'Content-Type': 'application/json' } },
  };
}

export function response(ctx) {
  if (ctx.error) {
    util.error(ctx.error.message, ctx.error.type);
  }
  const { statusCode, body } = ctx.result;
  if (statusCode === 404) {
    return null;                                  // unknown id → null, not an error
  }
  if (statusCode < 200 || statusCode >= 300) {
    util.error(`Backend returned HTTP ${statusCode}`, 'BackendError', null, { body });
  }
  return JSON.parse(body);                        // body is a string → parse to JSON
}
```

The `employees` resolver is identical except it calls `/employees`. Key points: the HTTP body must be **parsed**; transport errors and non-2xx responses are surfaced as clear GraphQL errors rather than a JavaScript crash; the APPSYNC_JS runtime does not support `encodeURIComponent`, so `util.urlEncode` is used.

11.3 Example query and results

```
query GetEmployees { employees { id name department email } }

→ {"data":{"employees":[{"id":"1","name":"Akash","department":"QA","email":"akash@example.com"},
                        {"id":"2","name":"John","department":"Engineering","email":"john@example.com"}]}}

query { employee(id: "999") { id name } }   → {"data":{"employee":null}}
```

11.4 Why AppSync sits between CloudFront and the backend

- **API façade and contract**: clients use one typed GraphQL schema and request exactly the fields they need; the REST backend can change without breaking clients.
- **Authentication at the edge of the backend**: API key (or Cognito/IAM/OIDC/Lambda authorizers) is enforced before any request reaches the ALB or ECS.
- **Aggregation**: resolvers can combine several backends/data sources into one response.
- **Managed and scalable**: no servers, built-in logging and metrics, request validation against the schema.
- CloudFront in front adds a global HTTPS edge, HTTP→HTTPS redirect, and a place for WAF and custom domains.

12 CloudFront

Setting	Value	Why
Origin domain	`4mhjvucnqjhlja7ekiqknqa7ey.appsync-api.us-east-1.amazonaws.com`	AppSync is the origin, as required
Origin path	*(empty)*	Avoids `/graphql/graphql`
Origin protocol	HTTPS only, TLS 1.2	Encrypted CloudFront→AppSync hop
Viewer protocol policy	Redirect HTTP to HTTPS	HTTP returns 301 to HTTPS
Allowed methods	GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE	GraphQL uses POST
Cache policy	**Managed-CachingDisabled**	GraphQL POST responses are dynamic and must not be cached
Origin request policy	**Managed-AllViewerExceptHostHeader**	Forwards `x-api-key`, content type and query strings, but sends AppSync's own Host header
Custom origin headers	None	Clients must send their own API key
Certificate	Default `*.cloudfront.net`	No custom domain needed

12.1 Verification

Request to `https://d1a7kv1va3r66w.cloudfront.net/graphql`	Result
POST with valid `x-api-key`	✓ 200 + employee data
POST without API key / wrong key	✓ 401 UnauthorizedException
HTTP instead of HTTPS	✓ 301 → HTTPS

12.2 Postman test

```
POST https://d1a7kv1va3r66w.cloudfront.net/graphql
Headers:  Content-Type: application/json
          x-api-key: <AppSync API key>
Body (raw, JSON):
{ "query": "query GetEmployees { employees { id name department email } }" }
```

13 Monitoring & alerting

13.1 Logs

Container stdout/stderr is sent by the `awslogs` driver to CloudWatch Logs group `/ecs/employee-task` (one stream per task, retention 7 days). Startup lines such as `Using postgres store` confirm the secret injection and database connection; failures are logged with the method, path and error message.

13.2 Dashboard `employee-api`

Widget	Metrics	Why
ECS CPU & Memory (%)	AWS/ECS CPUUtilization, MemoryUtilization (Average, 1 min) with an 80% line	Saturation / capacity
ALB requests & 5xx	RequestCount, HTTPCode_Target_5XX_Count, HTTPCode_ELB_5XX_Count (Sum)	Traffic and errors; separates app errors from ALB errors
ALB response time	TargetResponseTime p50, p95, p99	Latency as users experience it (percentiles, not averages)
Target health	HealthyHostCount (min), UnHealthyHostCount (max)	Is the service actually serving?
Alarms	Status of all three alarms	At-a-glance state
Recent application logs	Logs Insights query on `/ecs/employee-task`	Correlate metrics with log lines

13.3 Alarms (all notify SNS topic `employee-alerts` on ALARM and OK)

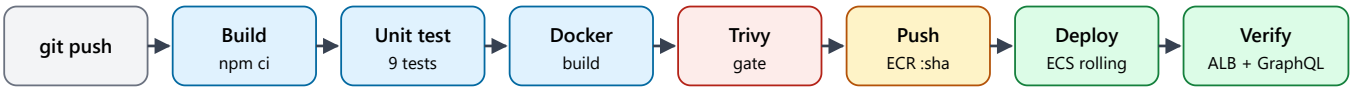
Alarm	Condition	Reasoning
`employee-ecs-cpu-high` required	ECS service CPUUtilization Average > 80% for 3 of 3 one-minute periods	Sustained saturation; 3 consecutive points avoid paging on a brief spike
`employee-unhealthy-targets`	UnHealthyHostCount ≥ 1 for 2 of 2 minutes	Capacity is degraded right now
`employee-5xx-errors`	HTTPCode_Target_5XX_Count > 5 per minute for 2 of 2 minutes; missing data = not breaching	Users are already seeing errors

Test command for the demo (forces the alarm state and sends the e-mail):

```
aws cloudwatch set-alarm-state --alarm-name employee-ecs-cpu-high \
    --state-value ALARM --state-reason "Demo: testing notification" --region us-east-1
```

> **Important**
> The SNS e-mail subscription must be confirmed (link in the "Subscription Confirmation" e-mail) or alarms fire without sending anything.

14 CI/CD pipeline (GitHub Actions)



Job 1 "Build & Unit Test" → Job 2 "Docker Build, Scan, Push & Deploy" (runs only if job 1 passes). AWS access via OIDC from the "Push" step onwards.

Figure 5 – Pipeline stages (.github/workflows/deploy.yml)

14.1 Workflow summary

Step	Implementation
Trigger	Push to `main`, or manual (`workflow_dispatch`); `concurrency` prevents overlapping deployments
Build / Unit test	`actions/setup-node@v4` (Node 22, npm cache) → `npm ci` → `npm test`
Docker build	`docker buildx build --provenance=false --sbom=false --load -t employee-api:${sha}`
Trivy scan	`aquasec/trivy:0.58.1 image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1` (pinned version)
AWS credentials	`aws-actions/configure-aws-credentials@v4`, `role-to-assume: ${{ vars.AWS_ROLE_ARN }}`; job permission `id-token: write`
Push to ECR	`aws-actions/amazon-ecr-login@v2` → tag and push `employee-api:${commit-sha}`
Deploy to ECS	Download current task definition → `amazon-ecs-render-task-definition@v1` (new image) → `amazon-ecs-deploy-task-definition@v2` with `wait-for-service-stability: true`
Verify	(1) service is running the new task definition ARN, (2) ALB `/health` returns UP, (3) GraphQL query through CloudFront returns `employees` and no `errors` (API key from `secrets.APPSYNC_API_KEY`)

14.2 GitHub OIDC → AWS (no stored credentials)

1. IAM identity provider: `token.actions.githubusercontent.com`, audience `sts.amazonaws.com`.
2. The job requests a signed OIDC token from GitHub (enabled by `permissions: id-token: write`).
3. AWS STS `AssumeRoleWithWebIdentity` validates issuer, audience and subject against the role's trust policy and returns credentials valid for about one hour.

Trust policy (role `github-actions-employee-api`)

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub":
      "repo:AkashAjayInApp@222200114/employee-management-api@1418440302:ref:refs/heads/main"
  }
}
```

The repository uses GitHub's **immutable subject** format, which embeds the numeric owner and repository IDs. Only the `main` branch of this exact repository can assume the role, and a different repository later created with the same name could not.

Permissions policy (`ecr-push-ecs-deploy`)

Statement	Actions	Resource
EcrLogin	`ecr:GetAuthorizationToken`	*
EcrPushEmployeeApi	BatchCheckLayerAvailability, InitiateLayerUpload, UploadLayerPart, CompleteLayerUpload, PutImage, BatchGetImage	repository/employee-api only
EcsDeploy	DescribeTaskDefinition, RegisterTaskDefinition, DescribeServices, UpdateService	*
PassExecutionRoleToEcsOnly	`iam:PassRole` with condition `iam:PassedToService = ecs-tasks.amazonaws.com`	role/ecsTaskExecutionRole

GitHub repository settings

Kind	Name	Content
Variable	`AWS_ROLE_ARN`	`arn:aws:iam::324440884610:role/github-actions-employee-api` (not sensitive)
Secret	`APPSYNC_API_KEY`	AppSync API key, encrypted and masked in logs

14.3 Result of the first successful run

All stages green (run 38521560781, attempt 3, commit `e207ac6`). The service moved to **task definition revision 5**, registered by `assumed-role/github-actions-employee-api/GitHubActions`, running image tag `e207ac649744…` on 2 healthy tasks. ECR scan on push: 0 findings. This deployment shipped the zlib and npm vulnerability fixes.

15 Security approach

Layer	Control
Edge	HTTPS via CloudFront (HTTP redirected); AppSync API key required (401 without it); API key kept out of code, screenshots and the README.
Network	Dedicated VPC; ECS and RDS in private subnets with no public IPs; NAT for outbound only; ECS not reachable from the internet (verified).
Security groups	Chained least privilege ALB → ECS → RDS using SG references; no 0.0.0.0/0 to 8080 or 5432.
Secrets	RDS-managed password in Secrets Manager; injected at task start via the execution role; only an ARN in the task definition; nothing in GitHub, Dockerfile, image or source.
Data	RDS encrypted at rest; TLS in transit with certificate verification against the RDS CA bundle; DB not publicly accessible.
IAM	Execution role can read exactly one secret; no task role (app needs none); CI role scoped to one ECR repo, ECS deploy actions, and PassRole only to ECS.
CI/CD	GitHub OIDC (no long-lived AWS keys); trust limited to one repo + main branch (immutable IDs); Trivy gate blocks fixable HIGH/CRITICAL CVEs; pinned scanner version.
Container	Non-root user, minimal Alpine base, OS patches applied at build, npm removed from runtime, ECR scan on push.
Source control	`.gitignore` excludes `.env`, `node_modules`, Terraform state and tfvars; the accidental home-directory git repository (which would have exposed `.aws` and `.ssh`) was replaced by a project-only repository.

16 Step-by-step build log

The infrastructure was first built quickly in the default VPC, then rebuilt properly to meet the networking requirements. The steps below describe the final setup in the order it was built.

Phase A – Application and container

1. Built the Express API with the required endpoints and `/health`.
2. Split `app.js` (routes) from `server.js` (startup/shutdown); added validation, SIGTERM handling and unit tests.
3. Wrote the hardened `Dockerfile`, `.dockerignore` and `.gitignore`; built and ran locally; verified non-root and graceful stop.
4. Created ECR repository `employee-api`; pushed versioned images (`1.0`, `1.1.0`).

Phase B – First deployment (default VPC) and the GraphQL/edge layer

1. Created ECS cluster, task definition and a 2-task service behind an ALB with `/health` checks.
2. Created the AppSync API with schema, HTTP data source and JS resolvers; recreated it as **public** after finding the first one was private.
3. Created the CloudFront distribution with AppSync as origin; fixed the Host header policy and origin path; removed the injected API key so clients must authenticate.
4. Demonstrated a zero-error rolling deployment (200/200 requests succeeded) and replaced the shared default security group with separate ALB and ECS groups without downtime.

Part 1 – VPC

VPC → Create VPC → **VPC and more**: name `employee`, 10.0.0.0/16, 2 AZs, 2 public + 2 private subnets, NAT **Zonal – in 1 AZ**, VPC endpoints none, DNS options on.

Part 2 – Security groups

EC2 → Security Groups: created `emp-alb-sg` (HTTP 80 from Anywhere-IPv4), then `emp-ecs-sg` (Custom TCP 8080, source = `emp-alb-sg`), then `emp-rds-sg` (PostgreSQL 5432, source = `emp-ecs-sg`), all in `employee-vpc`, one rule each.

Part 3 – RDS PostgreSQL

RDS → Create database → Standard create → PostgreSQL → db.t4g.micro, 20 GB gp3, autoscaling off → master user `postgres`, **Managed in AWS Secrets Manager** → VPC `employee-vpc`, new private DB subnet group, public access **No**, SG `emp-rds-sg` → initial database `employees`.

Part 4 – IAM permission for the secret

IAM → Roles → `ecsTaskExecutionRole` → Add permissions → Create inline policy → `secretsmanager:GetSecretValue` on the RDS secret ARN → name `read-employee-db-secret`. Verified with the policy simulator.

Part 5 – Database-backed application image

Added the PostgreSQL store (`pg`), TLS with the RDS CA bundle, table creation/seed at startup, pool error handling and an Express error handler. Tested two containers against one local PostgreSQL, including database outage and recovery. Built and pushed `employee-api:1.2.0` .

Part 6 – Target group and ALB

EC2 → Target Groups → IP addresses, `emp-tg` , HTTP 8080, `employee-vpc` , health `/health` (2/2/5 s/15 s, 200), no targets registered; Attributes → deregistration delay 30 s. EC2 → Load Balancers → Application Load Balancer `emp-alb` , internet-facing, both public subnets, SG `emp-alb-sg` , listener HTTP:80 → `emp-tg` . An HTTP 503 at this stage was expected (no targets yet).

Part 7 – Task definition revision

ECS → Task definitions → `employee-task` → Create new revision: image `1.2.0` ; environment `DB_HOST` , `DB_PORT` , `DB_NAME` (Value) and `DB_USER` , `DB_PASSWORD` (ValueFrom secret ARN `:username::` / `:password::`); execution role `ecsTaskExecutionRole` ; health check and logging retained.

Part 8 – ECS service in private subnets

ECS → `employee-cluster` → Create service: FARGATE, family `employee-task` revision 4, name `employee-api-service` , Replica, 2 tasks; rolling 100/200, circuit breaker + rollback; VPC `employee-vpc` , private subnets only, SG `emp-ecs-sg` , public IP off; existing ALB `emp-alb` , listener 80, target group `emp-tg` , grace period 60 s. Verified 2 healthy tasks, logs "Using postgres store", and shared data across tasks (10 × 200 for a newly created record).

Part 9 – Point AppSync at the new ALB

AppSync → Data sources → `employee_alb_source` → endpoint `http://emp-alb-820519526.us-east-1.elb.amazonaws.com` . Proven by creating a record directly in RDS via the new ALB and reading it back through CloudFront.

Part 10 – Remove the old setup

Deleted, in dependency order: old ECS service (force delete) → old ALB → old target group → old ECS security group → old ALB security group. Public IPv4 addresses dropped from 5 to 3; the ECS tasks have none.

Part 11 – Monitoring

Created the CPU > 80% alarm (3 of 3 minutes), two additional alarms, SNS topic `employee-alerts` with e-mail subscription, the `employee-api` dashboard, 7-day log retention and ECR scan on push. The scan surfaced the zlib CVE that was then fixed in the Dockerfile.

Part 12 – CI/CD

Created the GitHub repository, IAM OIDC identity provider, role `github-actions-employee-api` with the permissions policy, repository variable `AWS_ROLE_ARN` and secret `APPSYNC_API_KEY` ; pushed the code; fixed the OIDC subject mismatch; the pipeline then built, scanned, pushed and deployed revision 5 successfully.

17 Problems encountered & how they were fixed

These are real issues met during the build. Each one shows the symptom, how it was diagnosed, and the fix.

#	Symptom	Root cause / diagnosis	Fix
1	AppSync console query: *"Cannot read properties of undefined (reading 'data')"*	The API had been created with **Private** visibility; its hostname did not resolve publicly (DNS "name does not exist"), so the console got no response. Visibility cannot be changed after creation.	Recreated the API as **public** with the same schema; rewrote resolvers to parse the body and surface backend errors.
2	AppSync data source pointed at an API Gateway proxy	Inspection showed the HTTP data source was not the ALB, violating the required flow.	New data source pointing directly at the ALB; unused API Gateway deleted.
3	`employee(id)` always called `/employees`	Resolver ignored `ctx.args.id`.	Resolver calls `/employees/{id}`; 404 → `null`.
4	CloudFront returned a generic **403** page	Origin request policy **AllViewer** forwarded the viewer's `Host: …cloudfront.net`; AppSync routes by Host and rejected it. Origin path `/graphql` also doubled the path.	**AllViewerExceptHostHeader**; empty origin path.
5	Requests without an API key succeeded through CloudFront	The API key had been added as a **CloudFront custom origin header**, so CloudFront authenticated every request.	Removed the custom header; no key / wrong key now returns 401.
6	"Invalid JSON payload" / "Request body is empty"	Windows PowerShell 5.1 strips quotes from JSON passed to `curl.exe`; in Postman the body was not attached.	Body sent from a file / Bash; Postman Body → raw → JSON.
7	ECS tasks in public subnets with public IPs, sharing the ALB's security group	Initial console build in the default VPC.	New VPC with private subnets, separate SGs, public IP disabled; migrated without downtime.
8	Newly created employees "disappeared" ~50% of the time	In-memory storage: each of the 2 tasks had its own data; the ALB alternated between them.	Shared RDS PostgreSQL database.
9	App process crashed when the DB restarted	Unhandled `error` event from idle pool connections (found by stopping the DB in local tests).	`pool.on("error")` handler; app now returns 500 then recovers.
10	Non-root user could not read the CA bundle	`ADD --chmod=644` also applied 644 to the auto-created directory (no execute bit).	Bundle placed directly in `/app`.
11	Unexpected database cost	Aurora Serverless v2 created with min 2 / max 16 ACU and no auto-pause (~$6/day).	Recreated as RDS PostgreSQL db.t4g.micro to match the plan; Aurora cluster and snapshot deleted.
12	Env var `DB_PORT5432` in task definition	Value pasted into the key field during console entry.	New revision with key `DB_PORT`; faulty revision deregistered.

13	SNS topic named `employee-alb-sg`; e-mail never confirmed; dashboard showed only ECR pulls	Console entry mistakes found during review.	New topic `employee-alerts`, re-subscribed; dashboard rebuilt with ECS/ALB widgets.
14	ECR scan: HIGH CVE in zlib	Base image built before Alpine published `zlib 1.3.2-r1`.	`RUN apk upgrade --no-cache` in the Dockerfile.
15	Trivy: 10 HIGH CVEs in Node packages	All inside npm's own bundled dependencies, which the app never uses at runtime.	npm and corepack removed from the runtime image.
16	ECR could not scan image `1.2.0`	Docker Desktop pushed an OCI image index (with attestations).	CI builds with `--provenance=false --sbom=false`.
17	Pipeline: "*Not authorized to perform sts:AssumeRoleWithWebIdentity*"	CloudTrail showed GitHub sent the **immutable subject** `repo:Owner@id/repo@id:ref:refs/heads/main`; the console wizard wrote the legacy `repo:Owner/repo:…` form.	Trust policy updated to the exact immutable subject (StringEquals).
18	Verification step would not have failed on GraphQL errors	`actionlint`/shellcheck: a negated `! grep` does not trigger `set -e`.	Explicit `if grep …; then exit 1; fi`.
19	Git repository at the home directory	`C:\Users\…` was a git repo; `.aws` and `.ssh` were one `git add .` away from being committed.	Project-specific repository with a strict `.gitignore`.

18 Troubleshooting guide

18.1 Troubleshooting a 5xx through CloudFront → AppSync → ALB → ECS

Work from the outside in and test each hop on its own. The first hop that fails is where the problem is.

#	Hop	How to test / where to look	Typical causes
1	Client → CloudFront	Response headers `x-cache`, `x-amz-cf-id`; CloudFront error page wording; CloudFront metrics (4xx/5xx error rate)	Methods not allowed, wrong origin path, wrong origin request policy (Host header → 403), origin TLS problems (502)
2	CloudFront → AppSync	Call the AppSync endpoint **directly** with the same body and key. If direct works but CloudFront fails, the problem is CloudFront configuration.	Missing `x-api-key` forwarding, double `/graphql`, private AppSync API
3	AppSync → ALB	Enable AppSync logging (field resolver level ALL) and read `/aws/appsync/apis/<id>`: shows the HTTP request sent and the status/body returned	Wrong data source endpoint, ALB unreachable, resolver code errors, unparsed body, schema non-null violations
4	ALB	`curl http://<alb>/health`; target group **Targets** tab; metrics `HTTPCode_ELB_5XX` vs `HTTPCode_Target_5XX`; ALB access logs if enabled	**503** no healthy targets; **502** bad response/connection reset from target; **504** target timeout
5	ECS task	Service **Events** tab; **Stopped** tasks → stopped reason; CloudWatch Logs `/ecs/employee-task`	Unhealthy health checks, crash loops, image pull errors, secret retrieval errors, SG blocks 8080 from the ALB
6	ECS → RDS	App logs (timeouts, auth failures); RDS status and connections metric	SG blocks 5432, wrong host/name, secrets mis-mapped, DB stopped

The dashboard helps localise quickly: a rise in **HTTPCode_Target_5XX** points at the application, a rise in **HTTPCode_ELB_5XX** with low healthy hosts points at capacity/health, and no ALB traffic at all points upstream (AppSync/CloudFront).

18.2 Common issues and solutions

Issue	Cause	Solution
ALB returns 503	No healthy targets	Check target health; confirm `emp-ecs-sg` allows 8080 from `emp-alb-sg`; check the health path is `/health`; check tasks are running
Tasks stop with `ResourceInitializationError: unable to pull secrets`	Execution role lacks `GetSecretValue`, wrong ARN, or no outbound route	Fix the inline policy / ARN; confirm private route table 0.0.0.0/0 → NAT
Tasks stop with `CannotPullContainerError`	No route to ECR or tag does not exist	Check NAT route; confirm the tag exists in ECR
Logs: `connect ETIMEDOUT` to the DB	`emp-rds-sg` does not allow the tasks	Inbound 5432 from `emp-ecs-sg`

Logs: `password authentication failed`	Secret configured as Value instead of ValueFrom, or wrong JSON key	Use ValueFrom with `:username::` / `:password::`
CloudFront 403 "request could not be satisfied"	Viewer Host header forwarded to AppSync	Origin request policy AllViewerExceptHostHeader
GraphQL 401 Unauthorized	Missing/expired/wrong API key	Send `x-api-key`; rotate in AppSync → Settings → API keys
Deployment hangs then rolls back	New tasks never become healthy; circuit breaker triggered	Read stopped reasons and logs of the failed revision; fix and redeploy
Pipeline cannot assume role	Trust policy subject does not match the OIDC token	Compare with the `userName` in the CloudTrail `AssumeRoleWithWebIdentity` event
Pipeline fails at Trivy	A new fixable HIGH/CRITICAL CVE	Read the Trivy table; upgrade the package/base image; the gate is working as intended

19 Operations: demo, pause/resume, cost

19.1 Demonstration script

1. **Infrastructure**: show the VPC resource map, subnets/route tables, security group chain, ALB and target group (2 healthy), ECS service (2/2), RDS (private), dashboard and alarms.
2. **GraphQL query**: Postman POST to CloudFront with the API key → data; without the key → 401.
3. **Request flow**: explain CloudFront → AppSync → ALB → ECS → RDS using Figure 1.
4. **CI/CD**: change the message on `GET /` in `src/app.js`, `git commit -am "Update welcome message" && git push`, show the Actions run, the ECS deployment (old and new tasks side by side), and the new response.
5. **Self-healing**: ECS → service → Tasks → select one task → **Stop**. Watch Events: the service starts a replacement within about a minute; target health returns to 2 healthy. Keep Postman requests running – they keep succeeding on the remaining task.
6. **Alarm**: run the `set-alarm-state` command and show the e-mail.

19.2 Pausing to save cost

Action	Pause (in this order)	Resume (in this order)
1	ECS service desired tasks → **0**	RDS → Actions → **Start**; wait for **Available**
2	RDS → Actions → **Stop temporarily**	ECS service desired tasks → **2**; wait for 2 healthy targets

The app connects to the database at startup, so the database must be available before the tasks start. AWS restarts a stopped RDS instance automatically after 7 days. The NAT Gateway and ALB cannot be paused (only deleted).

19.3 Approximate running cost (us-east-1)

Item	≈ Per day
NAT Gateway (+ its public IP)	$1.08 + $0.12
ALB (+ 2 public IPs)	$0.54 + $0.24
2 Fargate tasks (0.25 vCPU / 0.5 GB)	$0.59
RDS db.t4g.micro + 20 GB (free tier if eligible)	$0.46
Secrets Manager, CloudFront, AppSync, ECR, CloudWatch at this traffic	≈ $0.02
Total while running	**≈ $3 / day**

20 Requirements traceability

#	Requirement	How it is met	Status
1	CloudFront → AppSync → ALB → ECS → DB	Sections 2, 9–12; proven end to end	Met
2	CRUD REST API + /health	Section 3	Met
3	Dockerfile, non-root, .dockerignore, local run	Section 4	Met
4	ECR + tagging strategy	Section 5	Met
5	ECS Fargate, ≥2 tasks, health checks, rolling deploy, recovery	Section 9	Met
6	ALB, target group, listener, /health, SGs	Sections 7, 10	Met
7	AppSync GraphQL via ALB, employees query	Section 11	Met
8	CloudFront entry point, cache/methods/headers, ECS not public	Section 12, 6	Met
9	VPC, public/private subnets, IGW, NAT, route tables	Section 6	Met
10	Least-privilege security groups	Section 7	Met
11	Terraform	The brief permitted manual configuration via the AWS Console; all resources were built manually/CLI and are fully documented here. Terraform is the first planned improvement (section 21).	Alternative
12	GitHub Actions with OIDC, all stages, deploy to ECS	Section 14	Met
13	Logs, ALB metrics, ECS CPU/memory, CPU > 80% alarm	Section 13	Met
14	Secrets Manager, no secrets in code	Section 8	Met
15	Documentation incl. ≥3 troubleshooting items	This document, section 18	Met

21 Known limitations & next improvements

Limitation	Improvement
Infrastructure created manually	Codify everything in Terraform (`vpc.tf` , `security-groups.tf` , `ecr.tf` , `ecs.tf` , `alb.tf` , `appsync.tf` , `cloudfront.tf` , `iam.tf` , `monitoring.tf`) with remote state in S3 + DynamoDB locking; import existing resources; run `plan` in CI on pull requests.
ALB is reachable directly from the internet (port 80, 0.0.0.0/0), so the REST API can bypass CloudFront/AppSync auth	Have the AppSync resolver add a secret header and configure an ALB listener rule to forward only when it matches (default action 403); or use a custom domain with an HTTPS listener (ACM) and restrict the source.
AppSync → ALB hop is HTTP	Add an ACM certificate and HTTPS:443 listener on the ALB (requires a custom domain).
Single NAT Gateway	One NAT per AZ, or VPC interface endpoints (ECR, Secrets Manager, Logs) + S3 gateway endpoint to remove the NAT dependency.
RDS single-AZ, 1-day backups	Multi-AZ, longer backup retention, deletion protection, automated secret rotation.
Fixed capacity of 2 tasks	ECS Service Auto Scaling (target tracking on CPU ~60% or ALB requests per target), validated with load tests.
GraphQL has queries only	Add mutations (create/update/delete) mapped to POST/PUT/DELETE.
API key authentication	Amazon Cognito or OIDC for user-level auth; AWS WAF with rate limiting on CloudFront.
Observability	Container Insights, AWS X-Ray tracing across AppSync/ALB/ECS, ALB and CloudFront access logs, structured JSON logging with request IDs.
Release safety	ECR tag immutability, signed images, blue/green deployments with CodeDeploy, CloudWatch-alarm-based rollback.

22 Interview questions & answers

Answers are written in the first person, based on what was actually built. They are meant as talking points, not scripts.

22.1 Architecture & design

Q. Walk me through the request flow of your application.

A client sends an HTTPS GraphQL POST to CloudFront. CloudFront forwards it (with the `x-api-key` header, but with AppSync's own Host header) to the AppSync API. AppSync validates the API key and the query against the schema, then runs a JavaScript resolver that turns the field into a REST call – for example `GET /employees` – against the HTTP data source, which is my ALB. The ALB forwards to one of two healthy ECS Fargate tasks in private subnets on port 8080. The Node.js app queries RDS PostgreSQL over TLS and returns JSON; the resolver parses it and AppSync returns only the requested fields back through CloudFront.

Q. Why put AppSync between CloudFront and the backend?

AppSync gives a typed GraphQL contract, authentication before anything reaches my VPC, and the ability to aggregate several backends behind one endpoint. Clients ask for exactly the fields they need. The REST service can evolve independently because resolvers map between the schema and the backend. CloudFront in front adds a global HTTPS edge, redirects HTTP to HTTPS and is the natural place for WAF and a custom domain.

Q. Why is caching disabled on CloudFront?

GraphQL requests are POSTs whose result depends on the body, and employee data changes. Caching them would risk serving stale or wrong data, so I used the CachingDisabled policy. If I needed caching, I would cache persisted queries sent via GET with the query hash in the cache key, or use AppSync's own server-side caching.

Q. Which origin request policy did you use and why?

AllViewerExceptHostHeader. I first used AllViewer, which forwarded `Host: …cloudfront.net`; AppSync routes requests by Host, didn't recognise it and returned 403. Forwarding everything except Host keeps `x-api-key` and the body while letting CloudFront send the AppSync hostname.

Q. How did you make sure ECS is not exposed to the internet?

Three layers: the tasks run in private subnets whose route table has no Internet Gateway route; `assignPublicIp` is disabled so they have no public address; and `emp-ecs-sg` allows port 8080 only from the ALB's security group. I verified it by calling a task IP directly – it timed out – while the same request through the ALB succeeded.

Q. What would you change for production?

- Terraform for all infrastructure with remote state.
- HTTPS on the ALB and stop direct access to it (secret header check from AppSync or private integration).
- WAF on CloudFront, Cognito instead of API keys.
- Multi-AZ RDS, NAT per AZ or VPC endpoints.
- Auto scaling, X-Ray tracing, blue/green deployments.

22.2 ECS & Fargate

Q. What is the difference between ECS and Fargate?

ECS is the container orchestrator – clusters, task definitions, services, scheduling, deployments and load balancer integration. Fargate is a serverless compute engine that ECS can run tasks on. With the EC2 launch type I would manage instances (AMIs, patching, capacity); with Fargate AWS provides the compute per task and I pay for the vCPU and memory each task requests.

Q. Why did you choose Fargate?

It's a small, stateless API with two tasks. Fargate removes server management and patching, gives each task its own network interface so security groups apply per task, isolates tasks from each other, and scaling is just the desired count. EC2 would only make sense for steady high utilisation, GPUs, or special host requirements.

Q. What is a task definition versus a service?

A task definition is an immutable, versioned blueprint: image, CPU/memory, ports, environment and secrets, health check, logging and IAM roles. Each change creates a new revision, which gives history and easy rollback. A service keeps a desired number of tasks from a revision running, registers them with the target group, replaces failed ones and performs deployments.

Q. How does ECS handle unhealthy tasks?

Two signals. If the container health check (the `wget /health` command) fails 3 times, ECS marks the task unhealthy and stops it. If the ALB health check fails 2 times, the target is marked unhealthy, the ALB stops routing to it, and ECS replaces the task. In both cases the service scheduler sees running < desired and starts a replacement, which is registered once it passes health checks. The 60-second grace period stops a slow-starting task from being killed during startup.

Q. How does your rolling deployment avoid downtime?

Minimum healthy 100% and maximum 200%: ECS starts the new tasks first, waits for them to be healthy in the target group, then drains the old ones (30-second deregistration delay) and sends SIGTERM. My app handles SIGTERM by finishing in-flight requests before exiting. I measured it: 200 requests during a deployment, all returned 200.

Q. What happens if a deployment is broken?

The deployment circuit breaker is enabled with rollback. If the new tasks keep failing to start or to become healthy, ECS marks the deployment failed and automatically returns to the last working revision. Because the old tasks are only stopped after the new ones are healthy, users keep being served by the old version meanwhile.

Q. What's the difference between the task execution role and the task role?

The execution role is used by the ECS agent before and around the container: pulling the image from ECR, writing logs to CloudWatch and fetching secrets. The task role is used by the application code itself to call AWS APIs. My app makes no AWS calls, so it has no task role; the secret permission is on the execution role.

Q. How did you size CPU and memory?

0.25 vCPU and 512 MB, the smallest Fargate size, because the API is lightweight. The dashboard shows CPU and memory per service; the CPU > 80% alarm tells me when it's under-provisioned. With my performance-testing background I would confirm the size with a load test – find the throughput where p95 latency or CPU degrades – and then set auto scaling with a target around 60% CPU.

22.3 Networking & security

Q. Why do you need a NAT Gateway?

The tasks are in private subnets with no public IPs, but at startup they must pull the image from ECR, fetch the secret from Secrets Manager and send logs to CloudWatch – all public AWS endpoints. The NAT Gateway in a public subnet gives outbound-only access. The alternative is VPC endpoints for those services, which also removes the NAT cost and dependency.

Q. You used one NAT Gateway – what's the risk?

It lives in us-east-1a. If that AZ fails, tasks in us-east-1b lose outbound access – they keep serving traffic coming in through the ALB, but a replacement task there couldn't pull its image. It was a deliberate cost trade-off for the assessment; production would have one NAT per AZ or VPC endpoints.

Q. Explain your security group design.

A chain where each layer only accepts traffic from the layer in front: internet → `emp-alb-sg` on 80; `emp-alb-sg` → `emp-ecs-sg` on 8080; `emp-ecs-sg` → `emp-rds-sg` on 5432. The sources are security group references, not CIDRs, so the rules stay correct as IPs change. Nothing opens the app or database port to 0.0.0.0/0.

Q. How are secrets handled?

RDS manages the master password in Secrets Manager, so nobody ever sees it. The task definition references the secret ARN with `ValueFrom` and the JSON key (`:username::` , `:password::`). At task start, the ECS agent uses the execution role – which can read only that one secret – to fetch the values and inject them as environment variables. Nothing sensitive is in GitHub, the Dockerfile, the image or the task definition. In CI, the AppSync key is a GitHub encrypted secret, masked in logs.

Q. Is the database connection encrypted?

Yes. RDS requires TLS and the app verifies the server certificate using the AWS RDS CA bundle baked into the image (`rejectUnauthorized: true`), so it can't be tricked into talking to an impostor. Storage is also encrypted at rest.

Q. What is a weakness in your current design?

The ALB is internet-facing with port 80 open so AppSync can reach it, which means someone who discovers the ALB DNS name can call the REST API directly, bypassing the API key. I'd close that by having the AppSync resolver add a secret header and making the ALB listener forward only requests with that header (everything else gets a fixed 403), plus HTTPS on the ALB. I prefer to call this out rather than hide it.

22.4 CI/CD

Q. Describe your pipeline.

On push to main: job 1 installs dependencies and runs the 9 unit tests. Job 2 runs only if that passes: builds the image tagged with the commit SHA, scans it with Trivy (fails on fixable HIGH/CRITICAL), assumes the AWS role via OIDC, pushes to ECR, renders a new task definition revision with the new image, deploys it to the ECS service and waits for stability, then verifies the service runs the new revision, the ALB health check is UP and a GraphQL query through CloudFront returns data.

Q. How does GitHub authenticate to AWS without access keys?

GitHub OIDC. The workflow has `id-token: write`, so it can request a signed token from GitHub. The `configure-aws-credentials` action calls `AssumeRoleWithWebIdentity`; AWS checks the token's issuer against the IAM identity provider, the audience (`sts.amazonaws.com`) and the subject (this repo, main branch) against the role's trust policy, then returns credentials that expire in about an hour. There are no long-lived keys to leak or rotate.

Q. Your first pipeline run failed – what happened?

"Not authorized to perform sts:AssumeRoleWithWebIdentity". The trust policy looked right, so I checked CloudTrail: the failed event showed GitHub was sending an **immutable subject** containing the numeric owner and repo IDs, while the console wizard had written the older `repo:owner/repo` format. I updated the trust policy to the exact subject. It's also more secure, because a re-created repository with the same name can't match.

Q. What least-privilege permissions does the CI role have?

ECR login, push only to the `employee-api` repository, the four ECS actions needed to deploy (describe/register task definition, describe/update service), and `iam:PassRole` only for `ecsTaskExecutionRole` and only when passed to `ecs-tasks.amazonaws.com`. It can't delete resources, read secrets, or touch RDS or IAM users.

Q. Why tag images with the commit SHA?

It's immutable and traceable: from any running task I can find the exact commit. Moving tags like `latest` can make a restarted task pull different code. Rollback is pointing the service to a previous task definition revision, which references a previous SHA.

Q. What did Trivy find and how did you handle it?

ECR found a HIGH CVE in zlib from the Alpine base; I added `apk upgrade` to pick up the patched package. Trivy then found 10 HIGH CVEs in Node packages – all inside npm's own bundled dependencies. Since the app runs with plain `node`, I removed npm and corepack from the runtime image. Result: 0 HIGH/CRITICAL, and the fix was shipped through the pipeline itself.

Q. How would you add Terraform to this pipeline?

A separate infrastructure workflow: `terraform fmt -check`, `init` with an S3 backend and DynamoDB lock table, `validate`, and `plan` posted as a pull-request comment; `apply` only on merge to main, with a protected environment requiring approval. State is never committed to Git; the app pipeline would keep deploying images to the ECS service Terraform creates.

22.5 Monitoring & troubleshooting

Q. How would you troubleshoot a 5xx from CloudFront?

Outside in, one hop at a time. First, is it CloudFront itself – look at the error page and the `x-cache` header. Then call AppSync directly with the same request; if that works, the issue is CloudFront config (methods, origin path, header policy). If AppSync fails, check AppSync logs to see what the resolver sent and got back. Then curl the ALB: 503 means no healthy targets, 502 a bad response from the app, 504 a timeout – the target group health and ALB 5xx metrics split ALB from app errors. Finally ECS service events, stopped-task reasons and the container logs, and the database connection from the logs.

Q. Why does your CPU alarm require 3 out of 3 data points?

To alert on sustained saturation, not single spikes. One minute above 80% during a burst is normal; three consecutive minutes means a real capacity problem. Fewer false alarms keeps people trusting the alerts.

Q. Why percentiles for latency?

Averages hide the slow tail. p95 and p99 show what the slowest 5% and 1% of users experience, which is what SLOs are written against – the same reason load-test reports use percentiles.

Q. Why doesn't /health check the database?

It's a liveness check that decides whether ECS kills a task. If it depended on the database, a 30-second DB blip would fail every task's health check and ECS would replace all of them at once, causing a full outage while the DB was down anyway. Database problems show up as 500s on data endpoints and in the 5xx alarm instead.

Q. Tell me about a bug you found by testing failure scenarios.

I stopped the database while the app was running. The process crashed, because the PostgreSQL pool emitted an `error` event for idle connections that nothing handled. In ECS that would crash every task at the same moment. I added a pool error handler; now the app returns 500 during the outage, `/health` stays up, and it recovers by itself when the database returns. The same test caught a file-permission bug that would have stopped every task from starting.

Q. How would you demonstrate ECS self-healing?

Stop one task from the console while sending requests. The service Events tab shows the replacement starting within about a minute, the target group goes back to two healthy targets, and requests keep succeeding on the remaining task because the ALB stopped routing to the stopped one.

22.6 Data, scaling & performance

Q. Why did you move from in-memory storage to RDS?

With two tasks, each had its own copy of the data. A POST landed on one task and the next GET might hit the other and return 404 – about half the time. It also lost all data on every deployment. A shared database fixes correctness and durability. I verified it: a record created through the ALB returned 200 on ten consecutive reads spread across both tasks.

Q. Why RDS PostgreSQL rather than Aurora Serverless?

I briefly created Aurora Serverless v2 with a minimum of 2 ACU, which cost about $6 a day idle. For this workload a db.t4g.micro is enough and much cheaper, so I recreated it as standard RDS. Aurora would make sense for higher availability needs and spiky load, with a sensible minimum capacity.

Q. How would you scale this service?

ECS Service Auto Scaling with target tracking – CPU around 60% or ALB request count per target – with min 2 and a sensible max. I'd load test to find each task's capacity and set the target and cooldowns from data. On the database side: connection pool sizing (each task has max 10), then read replicas or RDS Proxy if connections become the bottleneck.

Q. How does your performance-testing background help as a DevOps engineer?

I think in terms of throughput, latency percentiles, saturation and failure modes. In this project I used that to tune health checks and deregistration delay, to measure a deployment for errors with a probe, to build the dashboard around p95/p99 and 5xx, to set alarm thresholds that avoid noise, and to deliberately test failures – which found two real bugs before production.

22.7 Process & behaviour

Q. You didn't use Terraform – why, and what would it look like?

The assessment allowed manual configuration through the console, so I prioritised a working, secure system and thorough documentation. I would next codify it in the requested file structure, import existing resources with `terraform import` or import blocks, keep state in S3 with locking, and run plan/apply through CI. I'm aware that manual changes caused several issues here – a typo in an environment variable, a misnamed SNS topic – which is exactly what infrastructure as code prevents.

Q. What was the hardest problem?

The chain of small misconfigurations in the CloudFront → AppSync path: a private AppSync API, then the Host header 403, then a doubled origin path, then an API key injected by CloudFront that silently disabled authentication. What worked was testing each hop independently and comparing what was sent with what was expected – the same approach that later solved the OIDC trust issue via CloudTrail.

Q. How did you control cost?

Smallest sizes that meet the need, one NAT Gateway, removing the duplicate old environment, fixing the Aurora configuration, 7-day log retention, and a pause/resume routine – scale the service to 0 and stop RDS overnight, start RDS first in the morning. I tracked spend with Cost Explorer and recommended a budget alert.

End of document.